

SQL Constraints

The constraints are used to describe the rules for the data columns in a table. If there is any destruction between the constraints and the data action, the action is terminated immediately. The constraints make sure the dependability and the correctness of the data in the database.

In this section, we are going to discuss all the **Constraints**.

The constraints can be further divided as **column level or table level** where the **table level** constraints are used for the **whole table**, and the **Column level** constraints are used **only for one column**.

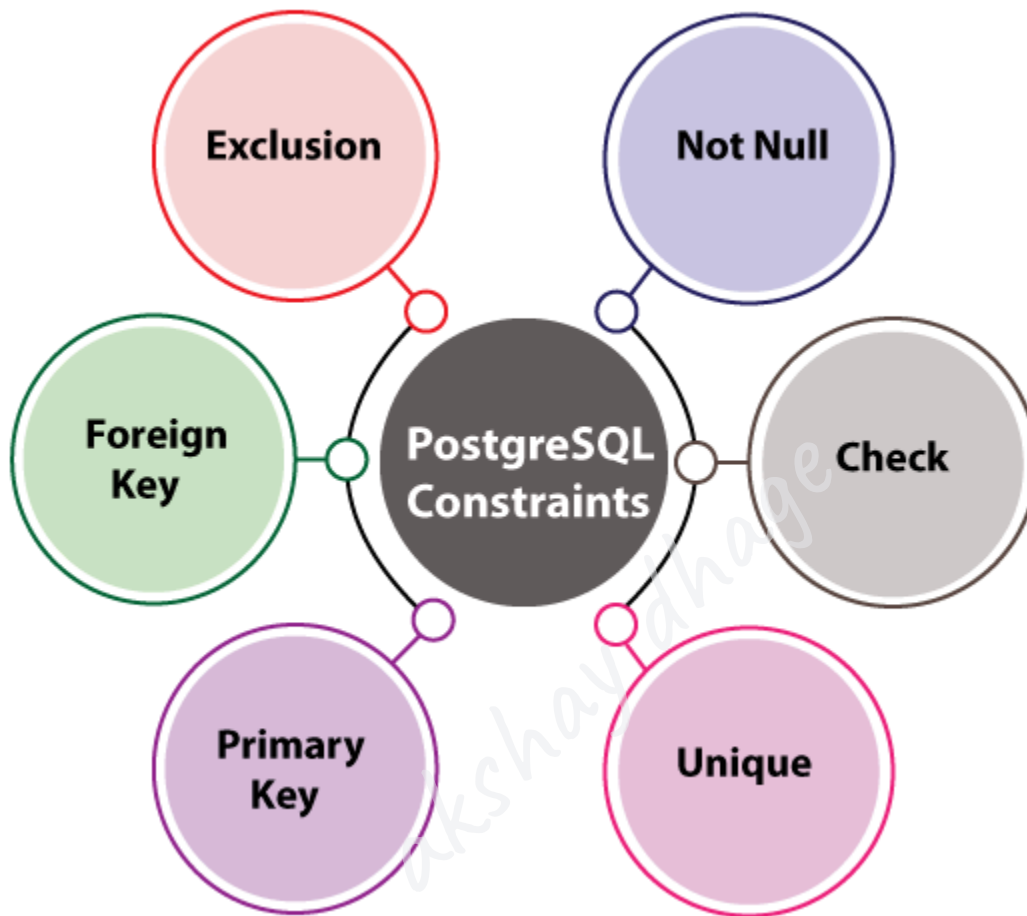
Where do we use the constraints?

The constraints are most commonly used in below areas:

- Once it is created, it can be added to a table, and we can also disable it temporarily.
- To individual columns, we can use the column constraints.
- When we are creating a table with the help of **creating table** command, we can also declare the constraints
- SQL can discard any value which interrupts the well-defined criteria.
- All the information related to the constraints is kept in the data dictionary.
- For one or more columns, we can use the table constraints.
- All the constraints are allocated a name.

Type of SQL Constraints

Let us see the most commonly used constraints in PostgreSQL:



Constraints	Description
Not Null	This type of constraint is used to make sure that a column cannot have a null value. And no name can be further defined to generate a not-null constraint.

Check	It is used to make sure that all values in a column or a field of a table satisfy particular situations such as it must match a Boolean expression. The Check constraint can be defined by a separate name.
Unique	The unique constraints are used to make sure that all values in a column of a table are exclusive.
Primary key	The primary key is to specify each row or record in a database table uniquely and make sure that there is no record duplicity in a particular table.
Foreign key	The foreign key is used to define that the value in a column or field of a table is equal to the real value of the primary key of another table.
Exclusion	This constraint is used to make sure that any two rows are linked on the precise columns or statements with the help of defined operators, and one of these operator assessments should return either Null or False .

NOT NULL Constraint

In not-null constraint, a column can hold the **Null** values by default. If we don't want a column to have a NULL value, then we need to explain such constraint on this column state that NULL is now not acceptable for that particular column. It is always created as a **column constraint**, and it represents unknown data but it doesn't mean that the data would be null.

For example

In the below case, we are creating a new table called **Customer**, which has the five columns, such as **Cust_Id**, **Cust_ Name**, **Cust_Address**, **Cust_Age**, and **Cust_Salary**.

```
CREATE TABLE Customer(  
  Cust_Id INT PRIMARY KEY NOT NULL,  
  Cust_Name TEXT NOT NULL,  
  Cust_Address CHAR(50),  
  Cust_Age INT NOT NULL,  
  Cust_Salary REAL  
);
```

Explanation

The above example shows the table **Customer** has been created in which **Cust_Id**, **Cust_Name**, and **Cust_Age** columns are specified not to accept the **null** values.

The table Structure

After executing the **select command**, we can see the table structure of the **Customer** table.

Data Output		Explain	Messages	Notifications		
	cust_id [PK] integer 	cust_name text 	cust_address character (50) 	cust_age integer 	cust_salary real 	

CHECK Constraint

In, the Check constraint can be defined by a separate name. It is used to control the value of columns being inserted. It allows us to verify a condition that the value being stored into a record. If the statement is false, then the data disrupts the constraint and is not saved into the table.

For example

In the below example, we are creating a new table called **Customer2**, and this table contains **five columns**.

```
CREATE TABLE Customer2(
  Cust_Id INT PRIMARY KEY NOT NULL,
  Cust_Name TEXT NOT NULL,
  Cust_Address CHAR(30),
  Cust_Age INT NOT NULL,
  Cust_Salary REAL check (Cust_Salary>0)
);
```

Explanation

In the above example, we add a **CHECK** with the **Cust_SALARY** column in table **Customer2**, where the **Cust_Salary** column can not contain any value less than or equal to **zero (0)**.

The table Structure

After executing the **select command**, we can see the table structure of the **Customer2** table.

18 **Select** * **from** Customer2;

19

Data Output

Explain

Messages

Notifications

	cust_id [PK] integer	cust_name text	cust_address character (30)	cust_age integer	cust_salary real

Unique Constraint

The **unique constraint** is used to maintain the individuality of the values that we store into a field or a column of the table. It is compatible with a group of column constraints or column constraints and a table constraint.

When we are using the **unique constraint**, an index on one or more columns generate automatically. If we add two different null values into a column in different rows, but it does not interrupt the specification of the **UNIQUE** constraint.

For example

In the below example, we are creating a new table called **Customer3**, which has similar five columns as we created in the above tables.

```
CREATE TABLE Customer3(
```

```
  Cust_Id INT PRIMARY KEY NOT NULL,
```

```
  Cust_Name TEXT NOT NULL,
```

```
  Cust_Address CHAR(30),
```

```
  Cust_Age INT NOT NULL Unique,
```

```
  Cust_Salary REAL default 20000.00
```

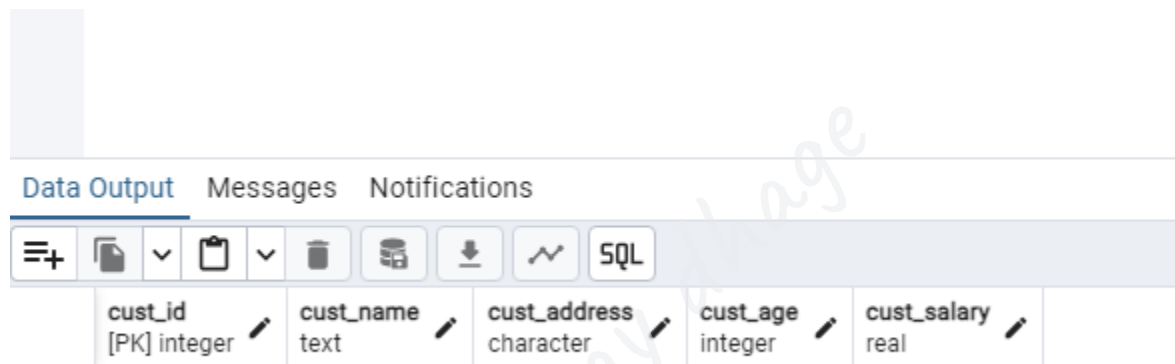
);

Explanation

The above example, the **Cust_Age** column, is set to **UNIQUE**; therefore, we can avoid the duplicity of two or more persons from having the same age.

The table Structure

After executing the **select command**, we can see the table structure of the **Customer3** table.



The screenshot shows a database management interface with a tab labeled 'Data Output'. Below the tab, there is a toolbar with icons for various actions. The table structure is displayed below the toolbar, showing columns and their data types.

cust_id	cust_name	cust_address	cust_age	cust_salary
[PK] integer	text	character	integer	real

Primary key Constraint

It is a field in a table that individually identifies each row or the record in a database table, and it contains a unique value. A primary key does not hold any null value. And for this, we can also say that the **primary key** is a collection of the **unique** and **not-null** of a table.

The working of the primary key is similar to a **unique** constraint. Still, the significant difference between them is one table can have only one **primary key**; however, the table can have one or more **unique** and **not-null** constraints.

Example

In the below example, we are creating a new table called **Employee**, which contains the **four columns**, such as **Emp_Id**, **Emp_Name**, **Emp_Address**, and **Emp_Age**.

```
CREATE TABLE Employee(  
  
Emp_Id INT PRIMARY KEY NOT NULL,  
  
Emp_Name TEXT NOT NULL,  
  
Emp_Address CHAR(30),  
  
Emp_Age INT NOT NULL  
  
);
```

Explanation

The above example, the **Employee** table, has been created with one **primary key** as **Emp_Id** column, which represents the **employee id** uniquely.

The table Structure

After executing the **Select command**, we can see the table structure of the **Employee** table.

```
8 Select * from Employee;
```

Data Output	Explain	Messages	Notifications
 emp_id [PK] integer 	emp_name text 	emp_address character (30) 	emp_age integer 

Foreign key Constraint

It is a group of columns with values that depend on the primary key benefits from another table. It is used to have the value in a column or group of columns that must be displayed in the same column or combination of columns in another table.

In SQL, the foreign key's values are parallel to actual values of the primary key in the other table; that's why it is also known as **Referential integrity Constraint**.

For example

In the below example, we are creating a new table called **Employee1**, which contains the four columns that are similar to the previous table.

```
CREATE TABLE Employee1(  
    Emp_Id INT PRIMARY KEY NOT NULL,  
    Emp_Name TEXT NOT NULL,  
    Emp_Address CHAR(30),  
    Emp_Age INT NOT NULL  
);
```

In this particular example, we will create one more table called **cust**, which contains three **columns**. And, here we create a **foreign key** as **Cust_Id** column, which references to the **Emp_ID** field for the **Employee1** table.

```
CREATE TABLE cust(  
    Emp_Id INT PRIMARY KEY NOT NULL,  
    Cust CHAR(50) NOT NULL,  
    Cust_Id INT references Employee1(Emp_Id)
```

);

The table Structure

Here, we can see the table structure of the **Cust** table, which is a reference to the **Employee1** table.

```
13 Select * from cust;
```

Data Output	Explain	Messages	Notifications
 emp_id [PK] integer 	cust character (50) 	cust_id integer 	

EXCLUSION Constraint

It is used to make sure that any two rows are linked on the specified columns or statements using the defined operators. In any case, one of these operator evaluations will return null or false.

For Example

In the below example, we are creating a new table called **Employee**, which contains the five columns. And here, we will use an **exclude** constraint as well.

```
CREATE TABLE Employee3(
```

```
  Emp_Id INT PRIMARY KEY NOT NULL,
```

```
  Emp_Name TEXT,
```

```
  Emp_Address CHAR(50),
```

```
  Emp_Age INT,
```

```
  Emp_SALARY REAL,
```

```
EXCLUDE USING gist (Emp_Name WITH =,Emp_Age WITH <>)  
  
);
```

Now, for **exclusion** constraints, →we will use the **using gist**, which is the index, and used for creating and implementation.

Note: If we are using the exclusion constraints, we have to run the create extension

btree_gist command, for one time in a database. And after that, it will connect the **btree_gist extension** that defines the constraints on basic scalar data types.

```
CREATE EXTENSION btree_gist;
```

Now, we will insert some records in the **Employee3** table, and we also imposed a similar age.

The first two **insert commands** will be executed successfully.

```
INSERT INTO Employee3 VALUES(101, 'john','Newyork',22,  
30000.00 );
```

```
INSERT INTO Employee3 VALUES(102, 'john','Florida',22, 30000.00  
);
```

And the records are added into the **Employee3** table as we can see in the below screenshot:

	emp_id [PK] integer	emp_name text	emp_address character (50)	emp_age integer	emp_salary real
1	101	john	Newyork	22	30000
2	102	john	Florida	22	30000

But for the **third insert** command, we may encounter the following error:

```
INSERT INTO Employee3 VALUES(103, 'john', 'Newyork', 32, 30000.00 );
```

ERROR: conflicting **key** value violates exclusion **constraint**

"employee2_emp_name_emp_age_excl" DETAIL: **Key** (emp_name, emp_age)=(john, 32) conflicts **with** existing **key** (emp_name, emp_age)=(john, 22).

10	<pre>INSERT INTO Employee3 VALUES(103, 'john', 'Newy</pre>		
Data Output	Explain	<u>Messages</u>	Notifications
<pre>ERROR: conflicting key value violates exclusion constraint "employee3_emp_name_emp_age_excl" DETAIL: Key (emp_name, emp_age)=(john, 32) conflicts with existing key (emp_name, emp_age)=(john, 22). SQL state: 23P01</pre>			

Dropping Constraints

If we want to delete a constraint, then we should remember the name of the constraints as it is easier for us to drop the constraints directly by its name. Otherwise, we will require to identify the system-generated name.

In the **psql**, the following command can be used to find out the names.

```
\d table name
```

The syntax for **dropping constraints** is as follows:

```
ALTER TABLE table_name DROP CONSTRAINT some_name;
```

Download Complete SQL & PostgreSQL Notes + Practice Files

You can access the full set of **SQL/PostgreSQL notes** along with practice datasets and queries from this GitHub repository:

👉 [SQL-resources-and-tutorials by akshay-dhage](#)